

`pick_place_node.cpp`

구현 강의 노트

ROS 2 + MoveIt 2 + C++ 기반 로봇 매니퓰레이터 소프트웨어 개발

대상: UR16e · MoveIt 2 · ros2_control · Gazebo

목차

- | | |
|---|--------------------------------|
| 1. Pick & Place 노드의 목적 | 11. IK 계산 및 Seed 전파 |
| 2. ROS 2 시스템 내에서 pick_place_node의 역할 | 12. 충돌이 없는 IK 해 선택 |
| 3. MoveIt 2를 이용한 매니퓰레이터 제어 흐름 | 13. 경로 계획 및 궤적 생성 |
| 4. 전체 코드 구조 개요 | 14. 그리퍼 Open/Close 제어 |
| 5. 노드 초기화 및 파라미터 설계 | 15. 궤적 실행 및 실패 처리 |
| 6. MoveGroupInterface 초기화 | 16. experiment_mode와 플래너 분기 구조 |
| 7. PlanningSceneMonitor 및 충돌 검사 | 17. 궤적 최적화 서버와의 통합 지점 |
| 8. Action 서버 기반 Pick/Place 명령 처리 | 18. 유닛 테스트 및 시뮬레이션 검증 전략 |
| 9. 현재 로봇 상태 읽기 | 19. 자주 발생하는 문제와 디버깅 방법 |
| 10. 목표 Pose 및 pre-grasp/pre-place Pose 생성 | 20. 전체 개발 로드맵 요약 |

1. Pick & Place 노드의 목적

본 노드는 두 개의 ROS 2 **Action 서버**(`/pick` , `/place`)를 외부에 제공합니다. 클라이언트는 대상 물체(또는 놓을 위치)의 6-DOF Pose(`base_link` 기준)를 보내고, 노드는 다음과 같은 전체 사이클을 수행합니다.

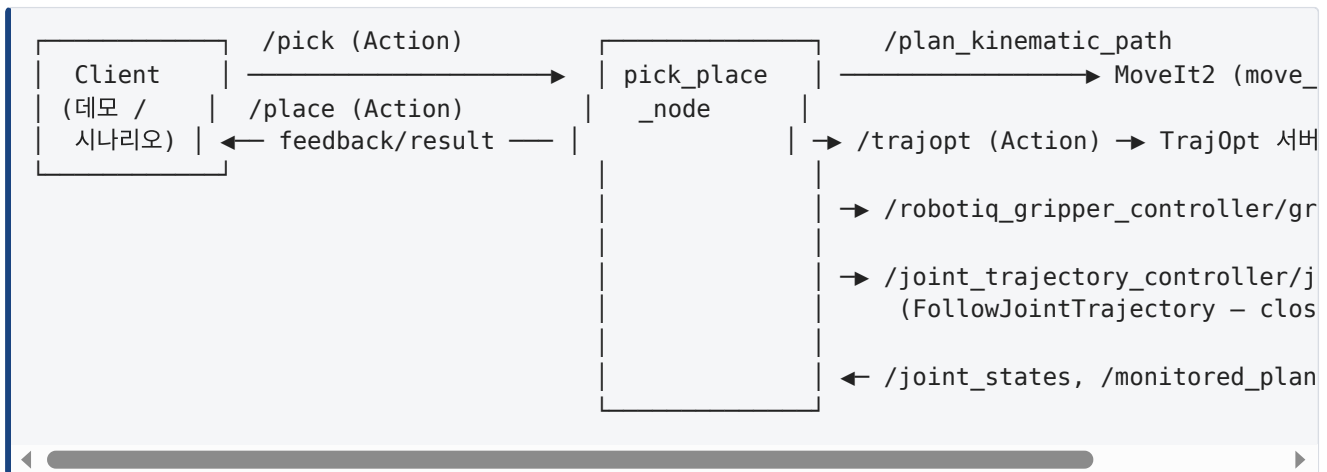
```
pre_grasp → grasp → lift → pre_place → place → retreat → home
```

설계 철학

1. **기하(Pose)와 동작(Motion)의 분리.** 모든 IK는 *모션 명령 이전에* 먼저 풀어둡니다. IK가 불가능하면 즉시 중단 — 반쯤 실행된 모션이 남지 않도록 합니다.
2. **pre-grasp → grasp 동일 IK 분기 유지.** grasp IK 해(solution)를 pre-grasp IK의 seed로 사용하여, 접근 중 팔꿈치가 뒤집히지 않도록 합니다.
3. **단일 인터페이스 + 3가지 플래닝 백엔드:** `rrt_only` , `trajopt_only` , `rrt_trajopt` . 외부 API는 동일하고, 런타임에 `experiment_mode` 파라미터로 전환합니다.
4. **충돌 인지 접근.** 후보 IK 자체와 강하 경로(joint-space 보간)를 모션 시작 전에 모두 충돌 검사합니다.

슬라이드 요약: pick-place 노드는 "단일 Pose 목표"를 "검증된, 충돌 없는, 최적화 가능한 관절 궤적"으로 변환하는 고수준 스킴이다.

2. ROS 2 시스템 내에서 `pick_place_node` 의 역할



- **상위(클라이언트):** 보통 시나리오 스크립트 또는 데모 컨트롤러가 pick action을 호출합니다.
- **하위:** `ros2_control(FollowJointTrajectory)`, Robotiq 그리퍼 action, 선택적으로 TrajOpt 서버.
- **측면(상태 공유):** `PlanningSceneMonitor` 가 `/monitored_planning_scene` 을 구독하여 `move_group` 과 동일한 월드를 본다.

슬라이드 요약: 본 노드는 오케스트레이터다. Pose Action을 소비하고, MoveIt 2 · TrajOpt · 그리퍼로 fan-out한 뒤 feedback/result로 응답한다.

3. MoveIt 2를 이용한 매니퓰레이터 제어 흐름

목표 Pose (base_link 기준)

- applyDownwardOrientation() ; tool Z = -world Z 강제, 현재 yaw 보존
- computeCollisionFreeIK() ; 목표 IK + self/world 충돌 검사
(anchor IK가 충돌 분기에 떨어지면 random seed restart)
- findCollisionFreePreApproachIK() ; pre-pose IK, seed = anchor solution
+ validateJointSpaceDescent() : joint-space 보간 충돌 검사
- planAndExecuteToJoints() ; experiment_mode 분기
 - └ rrt_only → MoveGroupInterface::plan + execute
 - └ rrt_trajopt → RRT plan → TrajOpt 서버에 초기 추정(seed)로 전달
→ 최적화 궤적을 execute (FollowJointTrajectory)
 - └ trajopt_only → RRT 미사용; q_start/q_end → TrajOpt 서버 (Hermite 초기화)
- approach (verticalCartesianMove | cartesianMove | joint plan)
grasp/release
retreat (best-effort)

슬라이드 요약: MoveIt 2는 블랙박스가 아니라 4가지 명시적 서비스(상태 조회, IK, PlanningScene 충돌, plan/execute)로 사용한다.

4. 전체 코드 구조 개요

레이어	역할	주요 엔티티
파일 스코프 헬퍼	L2 노름, 관절 순서 재정렬, 궤적 지표, CSV 로거	<code>computeL2Norm</code> , <code>reorderToUrOrder</code> , <code>TrajectoryMetrics</code> , <code>CsvLogger</code>
PickPlaceNode 생성자	파라미터, MoveGroup, PSM, 콜백 그룹, action 서버, 퍼블리셔, 워커 스레드	생성자
지오메트리 레이어	하향 orientation 계산, Pose 구성	<code>applyDownwardOrientation</code>
IK 레이어	Pose → 관절(seed 전파), 충돌 필터링	<code>computeIKForPose</code> , <code>computeCollisionFreeIK</code> , <code>findCollisionFreePreApproachIK</code>
충돌 레이어	PSM lock + 충돌 질의, 강하 경로 검증	<code>inCollision</code> , <code>validateJointSpaceDescent</code>
모션 레이어	plan/execute 분기, Cartesian 이동	<code>planAndExecuteToJoints</code> , <code>cartesianMove</code> , <code>verticalCartesianMove</code>
최적화 레이어	TrajOpt action 클라이언트 (RRT 사용/미사용)	<code>runWithTrajopt</code> , <code>runTrajoptOnly</code>
스킬 레이어	전체 Pick / Place 시퀀스	<code>executePick</code> , <code>executePlace</code>
동시성 레이어	워커 큐, abort 플래그, cancel 서비스	<code>workerLoop</code> , <code>enqueueTask</code> , <code>abort_requested_</code>
<code>main</code>	2-노드 부트스트랩(MoveGroup 노드 + PickPlace 노드, 별도 실행기)	<code>main</code>

슬라이드 요약: 본 파일은 계층적이다 — 헬퍼 → IK/충돌 → 모션/최적화 → 스킬. 각 레이어는 아래 레이어에만 의존한다.

5. 노드 초기화 및 파라미터 설계

학습 목표

`allow_undeclared_parameters(true)` 를 사용하는 이유, 각 파라미터 그룹의 역할, 그리고 파라미터를 **호출 시점마다 즉시(lazy)** 읽는 이유(외부 벤치마크 러너가 반복 사이에 플래너를 바꿀 수 있도록)를 이해합니다.

필요한 이론

- `rclcpp::NodeOptions().allow_undeclared_parameters(true)` — 생성자가 `declare_parameter` 를 끝내기도 전에 `set_parameters` 가 도착하는 레이스를 피한다.
- 파라미터 그룹: 기구학, 플래너 설정, 그리퍼, TrajOpt, 실험 모드, 접근 전략, 강하 검증.

스켈레톤

```
class PickPlaceNode : public rclcpp::Node {
public:
    explicit PickPlaceNode(std::shared_ptr<rclcpp::Node> mg_node)
        : Node("pick_place_node",
              rclcpp::NodeOptions().allow_undeclared_parameters(true))
    {
        declare_parameter<std::string>("arm_group", "ur_manipulator");
        declare_parameter<double>("pre_grasp_offset", 0.10);
        declare_parameter<std::string>("planner_id", "RRTConnectkConfigDefault");
        declare_parameter<bool>("use_trajopt", false);
        declare_parameter<std::string>("experiment_mode", "rrt_trajopt");
        declare_parameter<std::string>("approach_strategy", "vertical_cartesian");
        // ... pick_place_params.yaml 의 약 30여 개 파라미터
    }
};
```

검증 방법

`ros2 param list /pick_place_node` 및 `ros2 param set /pick_place_node planner_id RRTstarkConfigDefault`. 다음 plan 사이클의 CSV에 새 플래너가 기록되어야 합니다.

자주 발생하는 오류

- `allow_undeclared_parameters` 누락 → 러너가 시작 직후 크래시.
- 파라미터를 멤버에 캐싱 → 런타임 플래너 전환이 동작하지 않음.

슬라이드 요약: 선언은 관대하게, 읽기는 게으르게. 플래너는 재컴파일이 아니라 파라미터로 전환한다.

6. MoveGroupInterface 초기화

학습 목표

`MoveGroupInterface` 는 **별도 노드**에서 `SingleThreadedExecutor` 로 spin해야 한다는 점을 이해합니다.

왜 노드를 두 개로 분리해야 하는가

`MoveGroupInterface` 는 내부에서 동기 서비스 호출을 합니다. action 콜백과 같은 실행기를 사용하면, 콜백이 서비스를 기다리는데 그 서비스를 처리할 실행기가 콜백에 점유되어 데드락이 발생합니다.

스켈레톤 — main

```
int main(int argc, char ** argv) {
    rclcpp::init(argc, argv);
    auto mg_node = rclcpp::Node::make_shared("pick_place_move_group_interface");

    rclcpp::executors::SingleThreadedExecutor mg_exec;
    mg_exec.add_node(mg_node);
    std::thread mg_thread([&]{ mg_exec.spin(); });

    auto node = std::make_shared<PickPlaceNode>(mg_node);
    rclcpp::executors::MultiThreadedExecutor exec;
    exec.add_node(node);
    exec.spin();
}
```

스켈레톤 — 생성자 내부

```
move_group_ = std::make_shared<MoveGroupIface>(mg_node, arm_group);
move_group_ ->setPlannerId(planner_id);
move_group_ ->setMaxVelocityScalingFactor(0.3);
move_group_ ->setMaxAccelerationScalingFactor(0.3);
move_group_ ->setPoseReferenceFrame("base_link");
move_group_ ->setWorkspace(-2, -2, -2, 2, 2, 2);
```

검증 방법

`RCLCPP_INFO("Planning frame: %s", move_group_ ->getPlanningFrame().c_str());` 출력이 SRDF 설정에 따라 `world` 또는 `base_link` 로 표시되어야 합니다.

자주 발생하는 오류

- 단일 노드 구조 → 첫 `getCurrentState()` 에서 데드락.
- `setPoseReferenceFrame` 누락 → Pose 목표가 planning frame으로 잘못 해석됨.

슬라이드 요약: MoveGroupInterface는 자신만의 spin 실행기를 필요로 한다. 반드시 별도 노드 스레드로 분리한다.

7. PlanningSceneMonitor 및 충돌 검사

학습 목표

`move_group`이 사용하는 **동일한** `robot_description`과 `PlanningScene`을 재사용하여, 충돌 예측이 플래너의 충돌 판단과 일치하도록 합니다.

필요한 이론

- `PlanningSceneMonitor` (PSM)는 `/monitored_planning_scene` 과 `/joint_states` 를 구독한다.

- `LockedPlanningSceneRO ps(psm_)` 로 읽기 전용 스냅샷을 얻는다. 모션 호출 동안 락을 보유해서는 안 된다.
- `ps->checkCollision(req, res, state)` 가 SRDF self-collision matrix와 함께 FCL을 실행한다.

스켈레톤

```
psm_ = std::make_shared<planning_scene_monitor::PlanningSceneMonitor>(
    mg_node, "robot_description");
psm_->startSceneMonitor("/monitored_planning_scene");
psm_->startWorldGeometryMonitor();
psm_->startStateMonitor("/joint_states");
psm_->requestPlanningSceneState("/get_planning_scene");

bool inCollision(const std::vector<double>& q, const std::string& group) {
    if (!psm_) return false;
    planning_scene_monitor::LockedPlanningSceneRO ps(psm_);
    auto state = ps->getCurrentState();
    state.setJointGroupPositions(state.getJointModelGroup(group), q);
    state.update();
    collision_detection::CollisionRequest req; req.group_name = group;
    collision_detection::CollisionResult res;
    ps->checkCollision(req, res, state);
    return res.collision;
}
```

검증 방법

Gazebo에서 로봇 경로상에 장애물을 놓고 `inCollision()` 이 `true` 로 바뀌는지 확인합니다.

자주 발생하는 오류

- 별도의 `RobotModel` 을 생성 → SRDF self-collision matrix가 로드되지 않아 거짓 음성 발생.
- 락을 보유한 채 `move_group_->execute()` 호출 → 데드락.

슬라이드 요약: PSM은 빠르고 플래너와 일치하는 "이 관절 상태가 충돌하는가" 질의를 제공한다. 플래닝 전 IK 후보를 거를 때 사용한다.

8. Action 서버 기반 Pick/Place 명령 처리

학습 목표

`rclcpp_action::Server` 가 스레드를 **detach**하지 않고 큐에 **enqueue**하도록 구성하고, `cancel`을 지원하며, goal 상태에 맞는 종료 전이를 호출하도록 합니다.

필요한 이론

세 가지 콜백(`goal`, `cancel`, `accepted`) 중 함정은 `accepted` 입니다. 흔한 패턴인 `std::thread(...).detach()` 는 안전한 종료(use-after-free 방지)를 불가능하게 만듭니다.

스켈레톤

```
pick_server_ = rclcpp_action::create_server<Pick>(
    this, "pick",
    [this](const auto&, auto) { return rclcpp_action::GoalResponse::ACCEPT_AND_EXECUTE; },
    [this](auto gh) {
        abort_requested_.store(true);
        move_group_->stop();
        return rclcpp_action::CancelResponse::ACCEPT;
    },
    [this](auto gh) {
        enqueueTask([this, gh]{ executePick(gh); }); // detach()가 아님
    },
    rcl_action_server_get_default_options(), pick_cbg_);
```

워커 큐

```
void workerLoop() {
    while (true) {
        std::function<void()> task;
        {
            std::unique_lock<std::mutex> lk(queue_mutex_);
            queue_cv_.wait(lk, [&]{ return shutting_down_ || !task_queue_.empty(); });
            if (shutting_down_ && task_queue_.empty()) return;
            task = std::move(task_queue_.front()); task_queue_.pop();
        }
        try { task(); } catch (...) { /* 로그 */ }
    }
}
```

goal 상태 인지 종료 전이

```
auto abort_with = [&](const std::string& msg) {
    result->success = false; result->message = msg;
    if (gh->is_canceling()) gh->canceled(result);
    else gh->abort(result);
};
```

이유: CANCELING 상태의 goal에 `abort()` 를 호출하면 `IllegalTransitionException` 이 발생하여 SIGABRT 로 종료됩니다.

자주 발생하는 오류

- `std::thread(...).detach()` — 테스트에서는 멀쩡해 보이지만 종료 시 크래시.
- cancel을 단발 신호로 취급 — **긴 작업 전마다** 체크해야 한다.

슬라이드 요약: Action 서버는 detach 대신 enqueue. 종료 전이는 `is_canceling()` 에 따라 선택한다.

9. 현재 로봇 상태 읽기

학습 목표

`move_group_->getCurrentState(timeout)` 을 올바르게 사용하고, 적절한 관절 순서로 변환합니다.

필요한 이론

- `RobotState::copyJointGroupPositions(jmg, vec)` 는 `jmg->getVariableNames()` 순서로 반환하며, 이는 하드웨어 컨트롤러 순서와 다를 수 있다.
- `TrajOpt`에 보낼 때는 반드시 UR 컨트롤러 순서로 재정렬해야 한다.

스켈레톤

```
auto rs = move_group_->getCurrentState(2.0);
if (!rs) { /* 실패 처리 */ }
const auto* jmg = rs->getJointModelGroup(arm_group_name);
std::vector<double> q_moveit;
rs->copyJointGroupPositions(jmg, q_moveit);
auto q_ur = reorderToUrOrder(q_moveit, jmg); // 컨트롤러 순서
```

검증 방법

`ros2 topic echo /joint_states` 와 출력된 `q_ur` 를 비교 — 인덱스별로 일치해야 합니다.

자주 발생하는 오류

- 관절 순서가 SRDF 순서와 같다고 가정. **같지 않다.**
- 실행기 부하가 큰 상황에서 `getCurrentState` 를 timeout 없이 호출 — null을 조용히 반환한다.

슬라이드 요약: 관절 벡터는 암묵적 순서를 가진다. 컨트롤러나 `TrajOpt`에 보내기 전에 항상 하드웨어 순서로 재정렬한다.

10. 목표 Pose 및 pre-grasp/pre-place Pose 생성

학습 목표

(a) 현재 EE yaw를 보존하고 (b) tool-Z가 아래를 향하는 grasp Pose를 만들고, Z 오프셋만큼 위로 들어올린 pre-grasp Pose를 유도합니다.

필요한 이론 — `applyDownwardOrientation`

1. 현재 `base_R_ee` 를 구한다.
2. 현재 EE의 X축을 수평면에 투영한다.
3. `[x, y, -z_world]` 로 회전 행렬을 구성하여 quaternion으로 변환한다.
4. 상태 읽기에 실패하면 고정된 quaternion 파라미터로 폴백한다.

스켈레톤

```
geometry_msgs::msg::Pose grasp_pose = applyDownwardOrientation(goal->pick_pose);
geometry_msgs::msg::Pose pre_grasp_pose = grasp_pose;
pre_grasp_pose.position.z += pre_grasp_offset; // 예: 0.10 m
```

자주 발생하는 오류

- orientation을 하드코딩 → 매번 grasp마다 wrist가 불필요하게 회전.
- pre-grasp를 EE-Z 방향으로 오프셋 → EE가 기울었을 때 동작 실패. (불확실: 본 코드는 world Z 기준이며, tool Z 기준으로 하려면 world_R_ee 로 변환 필요.)

슬라이드 요약: pre-grasp = grasp의 world-Z 방향 들어올리기. orientation은 현재 yaw + 아래 방향 고정.

11. IK 계산 및 Seed 전파

학습 목표

pre-grasp와 grasp가 같은 IK 분기로 수렴하여 접근 중에 팔꿈치가 뒤집히지 않도록 합니다.

필요한 이론

- `RobotState::setFromIK(jmg, pose, timeout, validity_cb, opts, cost_fn)` 은 선택적 cost function과 함께 IK를 푼다(MoveIt 2 Humble 이상; 그 이하 배포판은 불확실).
- 같은 분기를 유지하는 두 가지 메커니즘:
 1. **Seed state** — 검색을 알려진 관절 구성에서 시작.
 2. **L2 cost** — $cost = w \cdot \sum (q_i - seed_i)^2$ 로 seed에 가까운 후보가 선택되도록 유도.
- 실패 시 random restart로 재시도 가능.

스켈레톤 — `computeIKForPose`

```
auto cost_fn = [w_l2, seed_state](
    const auto&, const moveit::core::RobotState& s,
    const moveit::core::JointModelGroup* jmg, const auto&) {
    std::vector<double> p;
    s.copyJointGroupPositions(jmg, p);
    return w_l2 * computeL2Norm(p, seed_state);
};

for (int att = 0; att < max_attempts && !found; ++att) {
    auto st = std::make_shared<moveit::core::RobotState>(*rs);
    if (att == 0) st->setJointGroupPositions(jmg, seed_state);
    else          st->setToRandomPositions(jmg);
    found = st->setFromIK(jmg, target_world, ik_timeout, {}, opts, cost_fn);
}
```

Seed 전파 패턴(핵심 설계)

```
// (1) grasp IK - seed = 현재 관절
computeIKForPose(grasp_pose, current_state, grasp_solution, ...);
// (2) pre-grasp IK - seed = grasp solution
computeIKForPose(pre_grasp_pose, grasp_solution, pre_grasp_solution, ...);
```

검증 방법

로그에서 `L2 vs seed` 를 관찰. $L2 > \pi$ 이면 다른 IK 분기에 떨어진 것 — 문제 있음.

자주 발생하는 오류

- pre-grasp를 먼저, grasp를 나중에 풀기 → 분기 일관성이 깨진다.
- `setFromIK` 에 planner의 world frame으로 변환하지 않은 Pose 입력(본 코드는 `world_T_base * target_in_base` 수행).

슬라이드 요약: seed 없는 IK는 그냥 샘플링이다. seed + L2 cost가 IK를 결정적 분기 선택기로 만든다.

12. 충돌이 없는 IK 해 선택

학습 목표

IK의 수렴 가능성과 안전성을 분리해서 다룹니다. 기구학적으로 유효한 IK도 self-collision일 수 있습니다.

본 파일의 두 함수

- `computeCollisionFreeIK` — anchor(grasp/place) Pose용. 충돌하는 해를 거부하고 random restart로 재시도.
- `findCollisionFreePreApproachIK` — pre-pose용. 추가로 후보 → anchor의 직선 **joint-space** 강하 경로를 N단계로 검증.

스켈레톤 — 강하 경로 검증

```
bool validateJointSpaceDescent(const std::vector<double>& a,
                              const std::vector<double>& b, int N) {
    for (int k=0; k<=N; ++k) {
        double alpha = double(k)/N;
        std::vector<double> q(a.size());
        for (size_t j=0; j<a.size(); ++j) q[j] = (1-alpha)*a[j] + alpha*b[j];
        if (inCollision(q, group)) return false;
    }
    return true;
}
```

핵심 설계 노트

방어적으로 **anchor 자체의 충돌 여부**를 먼저 점검한다. anchor가 콜리딩이면 모든 pre-approach 후보가 `alpha=1.0` 에서 실패하므로 무한 재시도가 된다.

자주 발생하는 오류

- IK 성공 = 안전, 으로 간주. 반드시 `inCollision` 으로 후처리 검증.
- `descent_check_steps` 를 너무 작게(≤ 2) 설정 → 중간 단계 충돌 누락.

슬라이드 요약: anchor 먼저, 경로 다음. IK 수렴성은 필요조건일 뿐 충분조건이 아니다.

13. 경로 계획 및 궤적 생성

학습 목표

IK가 고정된 이후에는 `MoveGroupInterface::plan()` 에 **관절 목표(joint target)**를 (Pose 목표가 아니라) 사용합니다.

필요한 이론

- `setJointValueTarget(joint_vec) + plan()` 은 OMPL의 `RRTConnect` (또는 설정된 `planner_id`)를 실행하고, 결과를 ITP/TOTG로 시간 파라미터화한다.
- 결과는 `execute()` 로 바로 사용 가능한 `moveit_msgs::RobotTrajectory` 이다.
- `setPlannerId()` 는 **plan 직전마다** 호출되므로 파라미터 변경이 한 사이클 내에 반영된다.

스켈레톤 — `planAndExecuteToJoints` (`rrt_only` 분기)

```
move_group->setPlannerId(get_parameter("planner_id").as_string());
move_group->setJointValueTarget(joint_target);

MoveGroupIface::Plan plan;
auto t0 = steady_clock::now();
if (!move_group->plan(plan)) { /* 실패 */ }
rec.rrt_planning_sec = durationSec(t0, steady_clock::now());
auto er = move_group->execute(plan);
```

Cartesian 변형

- `computeCartesianPath(waypoints, eef_step, jump_threshold, traj)` 는 0-1 사이의 *fraction*을 반환하며, `min_fraction` (보통 0.95) 미만이면 거부한다.
- `verticalCartesianMove` 는 `x, y, orientation` 을 끝점 Pose 값으로 고정하고 `z` 만 선형 보간한 N개의 waypoint를 생성한다 — joint plan이 보장하지 않는 직선 수직 강하를 강제한다.

자주 발생하는 오류

- IK를 이미 푼 다음에 `setPoseTarget` 을 또 호출 → IK가 중복 실행되고 분기가 달라질 수 있다.

- `computeCartesianPath` 에 `jump_threshold > 0` + 작은 `eef_step` → IK 경계에서 헛 실패.

슬라이드 요약: IK가 결정되면 관절 목표로 전환한다. Cartesian은 EE 기하 보장이 꼭 필요한 짧은 구간에만 사용한다.

14. 그리퍼 Open/Close 제어

학습 목표

`control_msgs/action/GripperCommand` 액션을 future 기반의 블로킹 래퍼로 구동합니다.

필요한 이론

- Robotiq 컨트롤러는 `/<name>/gripper_cmd ( control_msgs/action/GripperCommand )` 액션을 제공한다.
- 그리퍼 콜백 그룹은 반드시 **Reentrant**: pick/place 시퀀스가 arm 동작을 대기하는 동안에도 그리퍼 결과 콜백이 실행되어야 한다.

스켈레톤

```
bool controlGripper(double pos, double max_effort, double timeout) {
    auto promise = std::make_shared<std::promise<bool>>();
    auto future = promise->get_future();

    GripperCommand::Goal goal{};
    goal.command.position = pos; goal.command.max_effort = max_effort;

    rclcpp_action::Client<GripperCommand>::SendGoalOptions opts;
    opts.result_callback = [promise](const auto& res) {
        promise->set_value(res.code == rclcpp_action::ResultCode::SUCCEEDED);
    };
    gripper_client_->async_send_goal(goal, opts);
    return future.wait_for(seconds(timeout)) == future_status::ready && future.get();
}
```

자주 발생하는 오류

- 그리퍼 콜백 그룹을 `MutuallyExclusive` 로 설정 → pick action과 데드락.
- `goal_response_callback` 거부 케이스를 처리하지 않음 → promise가 set되지 않아 영원히 대기.

슬라이드 요약: 비동기 액션은 promise/future로 감싸고, 교차 호출되는 콜백은 Reentrant 그룹에 둔다.

15. 궤적 실행 및 실패 처리

학습 목표

궤적을 **closed-loop**로 실행하고, 모든 실패 모드를 명확하고 추적 가능한 라벨로 변환합니다.

필요한 이론 — closed-loop vs open-loop

- `move_group_->execute(traj)` 는 내부적으로 `FollowJointTrajectory`를 사용하며, 컨트롤러 완료까지 블로킹한다.
- 이전 구현은 `/joint_trajectory_controller/joint_trajectory` 로 publish 후 `sleep_for(T_opt + margin)` 이었다. 이는 *open-loop*이라 하드웨어 지연이 가정을 조용히 위반했다. Stage 1.1에서 `TrajOpt` 결과를 `RobotTrajectory` 로 감싸 `execute()` 로 전환했다.

실패 라벨 분리 원칙

Pick 시퀀스는 6가지 라벨을 분리해서 출력합니다.

- `grasp IK failed`
- `pre_grasp IK failed`
- `pre_grasp planning failed`
- `pre_grasp execution failed`
- `approach planning failed`
- `approach execution failed`

```
if (!pre_ok) {
    bool was_planning = pre_rec.message.find("planning") != std::string::npos;
    abort_with(was_planning ? "pre_grasp planning failed: " + pre_rec.message
                           : "pre_grasp execution failed: " + pre_rec.message);
    return;
}
```

Best-effort 단계

retreat(후퇴)은 best-effort입니다 — 경고만 남기고 abort하지 않습니다. 물체는 이미 잡혔거나 놓아졌기 때문입니다.

자주 발생하는 오류

- 모든 실패에 같은 "failed" 메시지를 반환 → CSV에서 triage 불가능.
- retreat 실패에 abort → 물체를 잡은 채로 공중에서 정지.

슬라이드 요약: 모든 실패는 단계+하위 단계로 라벨링한다. retreat은 best-effort.

16. `experiment_mode` 와 플래너 분기 구조

학습 목표

하나의 외부 함수(`planAndExecuteToJoints`)가 3개의 플래닝 백엔드로 분기하도록 만들어 단계별로 A/B/C 비교 실험을 가능하게 합니다.

모드 매트릭스

experiment_mode	파이프라인	폴백
rrt_only	setJointValueTarget → plan (RRTConnect) → execute	없음
trajopt_only	IK만 수행 → q_start/q_end (UR 순서) → TrajOpt 서버 → execute	없음
rrt_trajopt	RRT plan → TrajOpt(RRT를 초기 추정으로) → execute	TrajOpt 실패 시 RRT plan 폴백

스켈레톤(분기)

```
const std::string mode = get_parameter("experiment_mode").as_string();
move_group_>setPlannerId(get_parameter("planner_id").as_string());

if (mode == "trajopt_only") {
    auto q0 = reorderToUrOrder(current, jmg);
    auto q1 = reorderToUrOrder(joint_target, jmg);
    return runTrajoptOnly(q0, q1, step_name, rec);
}

MoveGroupIface::Plan plan;
move_group_>setJointValueTarget(joint_target);
if (!move_group_>plan(plan)) return false;

if (mode == "rrt_only") return bool(move_group_>execute(plan));

// rrt_trajopt
bool ok = runWithTrajopt(plan, true, current, joint_target, step_name, rec);
if (!ok) { rec.fallback_used = true; ok = bool(move_group_>execute(plan)); }
return ok;
```

검증 방법

CSV의 `planner_id`, `experiment_mode` 컬럼이 실제 사용된 값과 일치해야 하며, TrajOpt 실패 시 `fallback_used=true` 행이 보여야 합니다.

자주 발생하는 오류

- 모드는 바꿨는데 `setPlannerId` 를 누락 — 이전 플래너가 그대로 사용됨.
- `q_start/q_end` 를 MoveIt 변수 순서로 TrajOpt 서버에 보냄(서버는 UR 순서 기대).

슬라이드 요약: 모드는 파라미터 하나, 분기 하나, 백엔드 셋. 공정한 비교 실험에 최적이다.

17. 궤적 최적화 서버와의 통합 지점

학습 목표

RRT plan의 관절 waypoint를 추출하여 비선형 궤적 최적화기(/trajopt action)의 초기 추정으로 전달합니다.

필요한 이론

액션 인터페이스 pick_place_module/action/TrajOpt (불확실 — 정확한 .action 스키마는 repo 확인 필요)는 다음을 포함합니다.

- `q_start`, `q_end` (UR 순서)
- `num_waypoints`, `q_waypoints[]`, `timestamps[]` — flatten된 $K \times 6$
- `t_init`, `cheb_degree`, `use_reduced`, `use_free_t`
- 반환: `optimized_trajectory` (JointTrajectory), `t_opt`, `cost`, 토크 지표 등

스켈레톤 — RRT waypoint 추출

```
TrajOpt::Goal g;
g.t_init = t_init; g.cheb_degree = N_cheb;
g.use_reduced = use_reduced; g.use_free_t = use_free_t;
g.num_waypoints = jt.points.size();

for (auto& pt : jt.points)
    g.timestamps.push_back(pt.time_from_start.sec + pt.time_from_start.nanosec*1e-9);

for (auto& pt : jt.points)
    for (int j=0; j<6; ++j) {
        int k = joint_idx[j];          // UR → plan 순서 매핑
        g.q_waypoints.push_back(pt.positions[k]);
    }

g.q_start = q_start_ur; g.q_end = q_end_ur;
trajopt_client->async_send_goal(g, opts);
```

최적화 궤적의 closed-loop 실행

```
moveit_msgs::msg::RobotTrajectory rt;
rt.joint_trajectory = result->optimized_trajectory;
auto er = move_group_->execute(rt);
```

학생들이 흔히 하는 실수

- MoveIt 변수 순서의 waypoint를 UR 순서를 기대하는 TrajOpt에 그대로 전달 → 관절이 조용히 섞임.
- Hermite 전용 모드(`num_waypoints=0`)는 자유 공간에선 충분하지만, 장애물이 많은 장면에선 RRT 초기 추정이 필요.
- "NLP 미수렴"(`res->success == false`)을 실패로 간주. 본 파일의 정책은 경고 후 부분 해로 계속 진행입니다. (불확실 — 사용하는 TrajOpt 서버의 의미와 일치하는지 검증 필요.)

슬라이드 요약: RRT는 가능성, TrajOpt는 매끄러움. 두 단계를 잇는 다리는 plan을 액션의 q_waypoints 배열로 flatten하는 것이다.

18. 유닛 테스트 및 시뮬레이션 검증 전략

Stage A — 정적 검증 (로봇 없이)

- **파라미터 로드:** 런치 후 `ros2 param dump` 출력이 `pick_place_params.yaml` 과 일치해야 한다.
- **IK 라운드트립 테스트:** 도달 가능한 Pose에 대해 `applyDownwardOrientation` + `computeIKForPose` 를 호출하고, 결과 관절값으로 FK를 돌려 입력 Pose와 1 mm / 1° 이내로 일치하는지 단언.

Stage B — RViz 단독

- `experiment_mode=rrt_only` , `approach_strategy=joint` 로 설정.
- 로봇 베이스 앞 30 cm Pose로 `/pick goal` 전송. `/ee_path/planned` (Path 토픽)을 RViz에서 확인.

Stage C — Gazebo + MoveIt 2 + ros2_control

- UR16e + Robotiq 그리퍼를 Gazebo로 기동.
- 데모 스크립트로 pick/place 호출.
- 검증 항목:
 1. 접근 전에 그리퍼가 열리는가.
 2. RViz의 EE 궤적이 수직으로 곧게 내려오는가.
 3. 물체가 EE를 따라오는가(Gazebo attach 플러그인 또는 `moveit_msgs::AttachedCollisionObject`).

Stage D — 벤치마크 루프

- 러너가 시도 간에 `planner_id` 와 `experiment_mode` 를 토글.
- `~/ros/pick_place_exp/<timestamp>_results.csv` 를 확인하여 컬럼이 러너 의도와 일치하는지 검증.

자주 발생하는 오류

- IK 검증을 기구학적으로만 — 충돌은 PSM에서만 보이며, PSM은 `startSceneMonitor` 후에만 동작.
- `ros2_control`이 로드되지 않은 상태에서 RViz의 OMPL 성공을 신뢰 — `execute` 는 조용히 실패한다.

슬라이드 요약: 정적 → RViz → Gazebo → 벤치마크. 각 단계가 서로 다른 버그 클래스를 잡는다.

19. 자주 발생하는 문제와 디버깅 방법

증상	가능한 원인	확인 지점
<code>getCurrentState failed</code>	joint state 수신 지연	<code>/joint_states</code> 발행 주기 확인, timeout 인자 상향
분명히 도달 가능한 Pose인데 IK FAILED	Pose가 잘못된 frame	<code>applyDownwardOrientation + world_T_base * target_in_base</code>
IK는 OK이지만 <code>descent COLLISION at step 0/N (alpha=0)</code>	pre-pose IK 자체가 충돌	<code>findCollisionFreePreApproachIK</code> random seed 재시도, <code>descent_max_ik_retries</code> 상향
<code>descent COLLISION at step N/N (alpha=1.0)</code>	anchor IK 자체가 충돌	anchor용 <code>descent_max_ik_retries</code> 상향 또는 목표 Pose 수정
Cartesian path 30 %	<code>eef_step</code> 이 너무 큼 또는 Pose가 직선 도달 불가	<code>cartesian_eef_step</code> 을 0.005로, 여전히 실패면 <code>joint</code> 전략 사용
<code>TrajOpt server not available</code>	<code>/trajopt</code> 가 미실행	<code>ros2 action list grep trajopt</code> , <code>trajopt_server_timeout_sec</code> 상향
종료 시 노드 크래시(SIGABRT)	goal 상태와 종료 전이 미스매치	<code>executePick/Place</code> 가 <code>is_canceling()</code> 분기를 쓰는지 확인
pre-grasp ↔ grasp에서 팔꿈치 뒤집힘	seed 전파 누락	pre-grasp IK seed가 <code>grasp_solution</code> 인지(현재 상태 아님) 확인

슬라이드 요약: 실전에서의 대부분의 실패는 플래너 자체가 아니라 IK/충돌의 오분류다. 실패 라벨을 먼저 본다.

20. 전체 개발 로드맵 요약

- 스켈레톤 노드 — 파라미터 선언, 2-노드 `main`, 빈 action 서버.
- MoveGroupInterface** — 초기화, planner/scaling/frame/workspace 설정.
- PlanningSceneMonitor** — 3개 모니터 기동, `inCollision()` 구현.
- 그리퍼 클라이언트 — Reentrant CBG + future 기반 `controlGripper`.
- 워커 큐 기반 action 서버 — `enqueueTask`, `workerLoop`, abort 플래그, cancel 서비스.
- applyDownwardOrientation** — 현재 yaw 보존, tool-Z 하향 강제.
- computeIKForPose** — seed + L2 cost IK (base → world 변환 포함).
- 충돌 인지 IK — `computeCollisionFreeIK`, `validateJointSpaceDescent`, `findCollisionFreePreApproachIK`.
- planAndExecuteToJoints** — `rrt_only` / `rrt_trajopt` / `trajopt_only` 단일 분기.
- Cartesian 헬퍼** — `cartesianMove`, `verticalCartesianMove`.
- TrajOpt 클라이언트** — `runWithTrajopt` (RRT seed), `runTrajoptOnly` (Hermite).
- executePick / executePlace** — 7단계 시퀀스 + cancel 체크포인트 + 라벨링된 실패.
- CSV + EE Path 퍼블리셔 — 벤치마크용 관찰성.
- Gazebo + 벤치마크 검증 — 플래너/모드 sweep, CSV 분석.

강의 노트 끝 — pick_place_node.cpp